第7课 线性 Attention：从状态累积到 KDA

MLA 压缩每条历史记录，稀疏 Attention 选择部分历史位置；它们仍保留逐 token 的历史表示。**线性 Attention 把历史不断写入一份固定大小的状态，新的 Query 直接读取这份状态。**

固定状态减少了历史扫描与存储，但也会产生记忆干扰。DeltaNet 用预测误差纠正已有记忆，Gated DeltaNet 与 KDA 引入遗忘机制，分块计算让状态更新能够高效执行。本课沿这些机制的演进，连接到 vLLM 中的状态缓存与 Prefill、Decode 执行路径。

一、线性 Attention 的基本原理

两种 Attention 的基础公式

当前位置 t 可以读取位置 0 到 t 的 Key、Value。将它们分别按行排成矩阵 K、V，当前 Query 记作 q_t ，Key、Value 的维度分别记作 d_k, d_v 。标准 Attention 先计算 Query 与各个 Key 的点积，缩放后用 Softmax 得到权重，再加权 Value：

$$o_t^{\text{standard}} = \text{softmax}\left(\frac{q_t K^\top}{\sqrt{d_k}}\right) V.$$

先看最基础的归一化点积形式的线性 Attention：直接把点积作为权重，再除以权重总和。下面的例子采用非负的 Q/K，且权重总和大于零：

$$o_t^{\text{linear}} = \frac{\sum_{j=0}^{t}(q_t k_j^\top) v_j}{\sum_{j=0}^{t} q_t k_j^\top} = \frac{q_t\left(\sum_{j=0}^{t} k_j^\top v_j\right)}{q_t\left(\sum_{j=0}^{t} k_j\right)^\top} = \frac{q_t S_t}{q_t z_t^\top}.$$

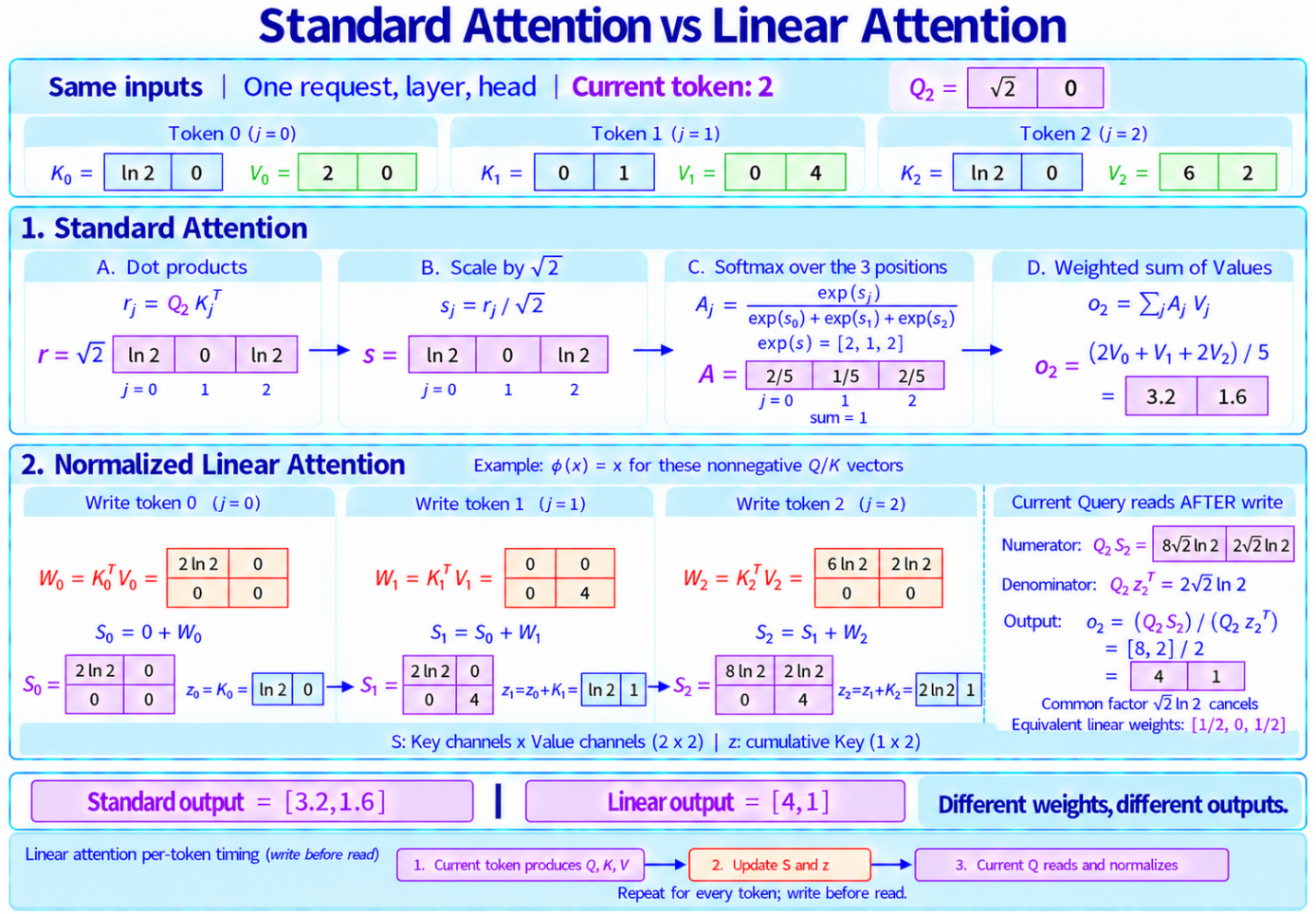
对于当前 Query， q_t 不随历史位置 j 改变，可以提到求和号外。**S 是 K/V 外积的累计矩阵，z 是 Key 的累计向量。** 外积 $k_j^\top v_j$ 是 Key 列向量乘 Value 行向量，矩阵中的每个元素都是一个 Key 分量与一个 Value 分量的乘积。

这两份状态都不依赖当前 Query，可以随 token 到来不断累计；新的 Query 读取它们，分别得到 Value 加权和与权重总和，再相除得到输出。这里的 Query 是行向量，分子是向量、分母是标量，不能把上下的 Query 约掉。

如果先计算 $qK^\top$，就要读取所有历史 Key，得到逐位置的权重，再用这些权重加权对应的 Value。**沿这条计算路径，历史 K/V 仍需按位置保留，不能只靠一份合并后的状态完成计算。**

同一组输入，两种计算方式

图中使用同一组 Q/K/V 对比两种 Attention。下标 0、1、2 表示 token 位置；W 是单个 token 的 K/V 外积， $S0$ 到 $S2$ 、 $z0$ 到 $z2$ 表示累计状态随写入逐步更新。两边的权重规则不同，最终输出也不同。



为什么称为“线性”

逐位置计算 Attention 时，第一个位置匹配一条 Key，第二个位置匹配两条，直到第 N 个位置匹配 N 条。整个序列需要 $N(N+1)/2$ 次 Query-Key 匹配，随序列长度按平方增长。

状态读取的工作量取决于状态大小：S 的形状始终为 $d_k \times d_v$，z 的长度始终为 d_k。每个 token 的更新与读取成本为 $O(d_k d_v)$，处理整个序列的成本为 $O(N d_k d_v)$。**Key、Value 的维度固定时，计算量随序列长度线性增长，这就是“线性”的含义。**

标准 Attention 的 Softmax 包含指数运算，不能像上面的点积权重一样，将 Query 提出来并精确合并成 S、z。

固定状态会丢掉什么

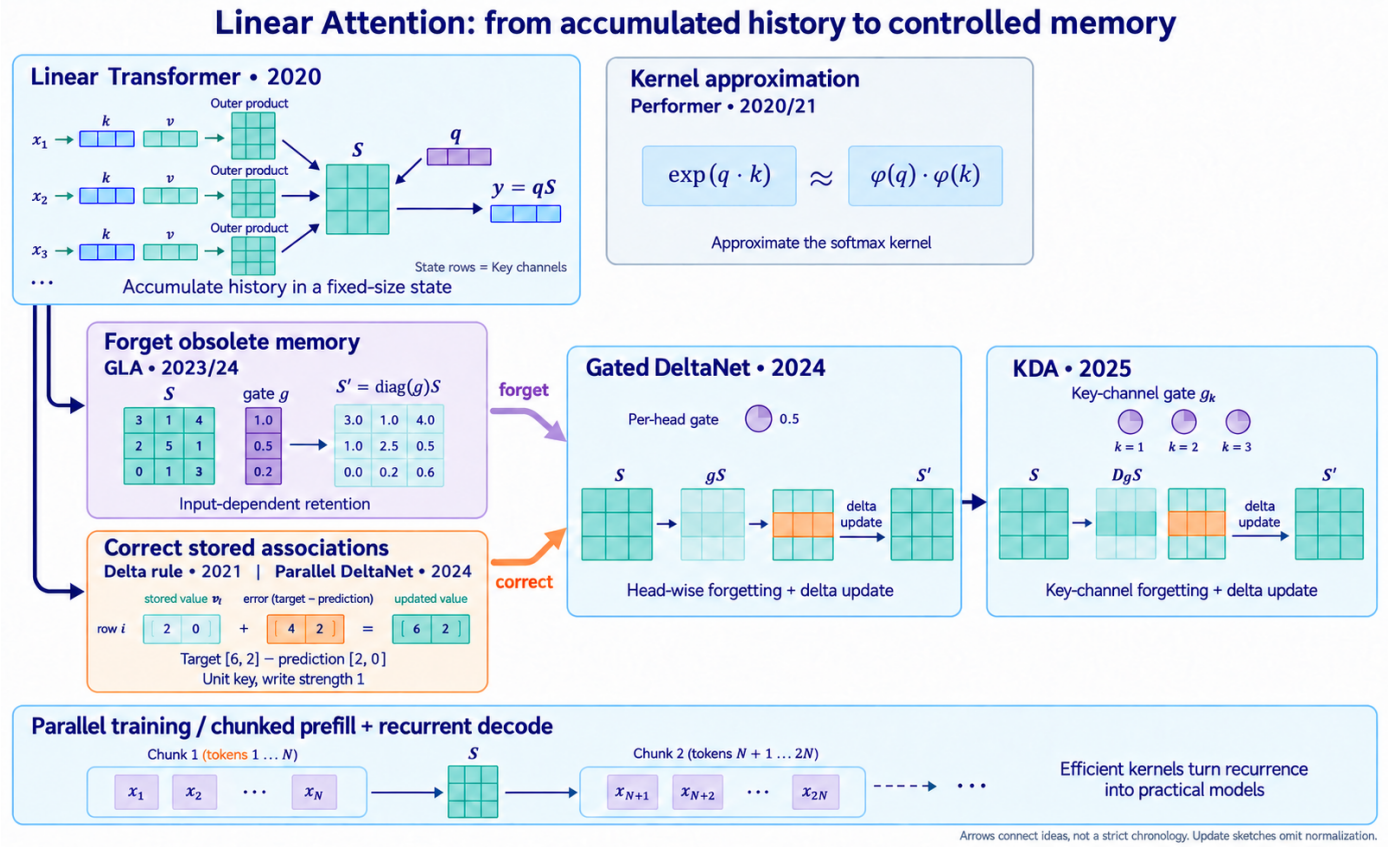
图中位置 0、2 的 Key 相同，写入的 Value 分别是 [2,0]、[6,2]。在线性路径中，当前 Query 对这两次写入赋予相同权重，输出是平均值 [4,1]。如果这两次写入表示“同一条关联从旧内容 [2,0] 更新为新内容 [6,2]”，简单累加就无法完成替换：旧贡献仍留在状态中。

所有历史又共享同一份矩阵。不同 Key 的方向如果重叠，写入一条关联也会影响其他关联的读取。合并后的状态无法像 KV Cache 那样，直接按历史位置取回那条独立的 K/V。

这就引出了两类改进：**纠错，让新内容能够修正旧关联；遗忘，让模型控制旧记忆保留多少。**

相关工作

下图汇总了几项代表性工作：Linear Transformer 将历史累积进固定状态；Performer 用随机特征近似 Softmax 权重；GLA 用门控控制旧记忆的保留比例；Delta rule 根据预测误差纠正已有关联；Gated DeltaNet 结合遗忘与纠错；KDA 将遗忘细化到每个 Key 通道。DeltaNet 的并行算法 以及图底部的分块计算，则让状态更新能够在训练和 Prefill 中高效执行，在 Decode 中逐步递推。



二、DeltaNet：用预测误差更新记忆

DeltaNet 用当前 Value 与旧状态的计算结果之间的差额，修正状态。 核心公式是：

$$S_t = S_{t-1} + \beta_t k_t^\top (v_t - k_t S_{t-1}),$$
$$o_t = q_t S_t.$$

这里的 Q/K/V 都是行向量，S 的形状为 $d_k \times d_v$ 。 k_t 是当前 Key， S_{t-1} 是本次更新前的状态； β_t 控制采用多少差额，由当前输入产生。DeltaNet 论文 §2.2

单位 Key：外积写入与伪逆读取

先只保存一条关联。外积 $k^\top v$ 将 Value 的每个分量变成状态的一列；用同一个 Key 乘状态，就是分别对这些列做点积。Key 长度为 1 时：

$$S = k^\top v,$$
$$kS = (kk^\top)v = \|k\|^2 v = v.$$

这里 k 是写入矩阵 $k^\intercal$ 的伪逆：它能还原沿 Key 方向写入的 Value。 例如 $k = [0.6, 0.8]$ 、 $v = [5, 10]$ ：

$$S = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix} [5, 10] = \begin{bmatrix} 3 & 6 \\ 4 & 8 \end{bmatrix},$$

$$kS = 0.6[3, 6] + 0.8[4, 8] = [5, 10].$$

读取只取出状态各列沿 Key 方向的分量，不能还原整个状态。多条关联写入后，其他 Key 也可能影响这个结果。因此 $k_t S_{t-1}$ 是旧状态对当前 Key 的预测，并不保证等于某次写入的原始 Value。

差额如何修正状态

$v_t - k_t S_{t-1}$ 是当前目标与已有预测的差。将差额乘 $k_t^\top$ ，就是沿当前 Key 的方向修改状态的各列。对单位 Key，更新后的预测满足：

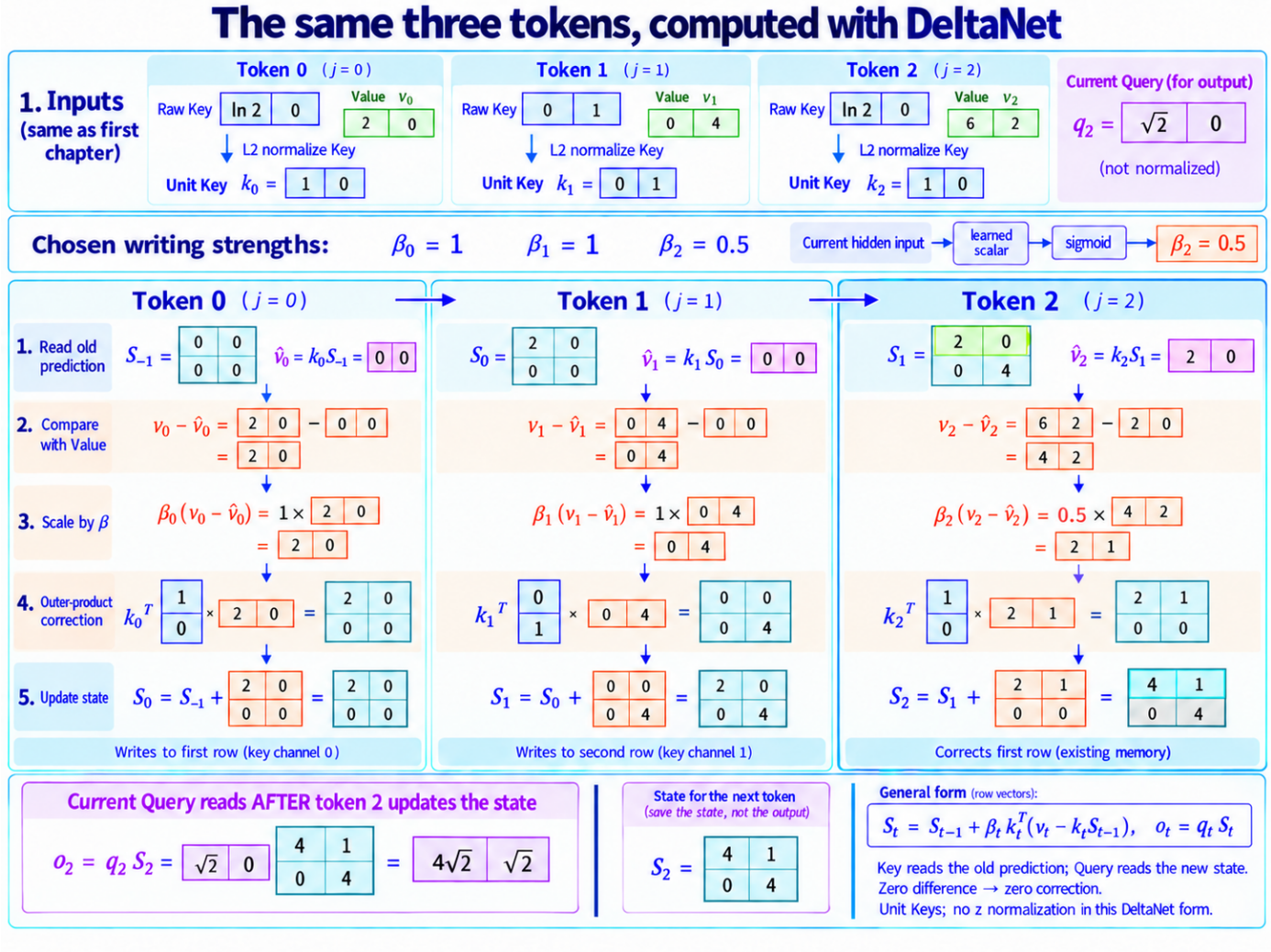
$$\begin{aligned} k_t S_t &= k_t S_{t-1} + \beta_t (k_t k_t^\top)(v_t - k_t S_{t-1}) \\ &= k_t S_{t-1} + \beta_t (v_t - k_t S_{t-1}) \\ &= (1 - \beta_t) k_t S_{t-1} + \beta_t v_t. \end{aligned}$$

β＝1 时，当前 Key 的新预测等于目标 Value；β＝1/2 时，移动到旧预测与目标的中点。 差额为零时，状态不变。Key 不归一化时，读回的修正量还会乘上其长度的平方。

为什么输出直接用 qS

第一章先累加各个 Value 的加权贡献，再除以权重总和，得到加权平均。DeltaNet 的更新则已经在当前 Key 对应的预测上完成了比例混合。上面的推导展开后，旧预测的系数是 $1 - \beta_t$ ，当前 Value 的系数是 β_t ： **两者加起来就是 1，所以这个混合结果不用再除一次权重总和。**

图沿用第一章的 Q/K/V，将 Key 归一化，β 依次取 1、1、1/2；Query 仍为 `[√2,0]`。从零状态执行三次更新，最后读取新状态：



Gated DeltaNet：先衰减旧状态，再纠错

DeltaNet 沿当前 Key 的方向修正预测，与该 Key 正交的旧信息仍然保留。当上下文切换时，这些信息也可能需要清除。Gated DeltaNet 增加了衰减系数 α_t：先把旧状态整体乘以 α_t，再执行 Delta 修正。

$$S_t = \alpha_t S_{t-1} + \beta_t k_t^\top \left(v_t - k_t \alpha_t S_{t-1}\right).$$

差额根据衰减后的状态计算。 α_t 控制旧状态保留多少，β_t 仍控制差额修正多少；它们各自由当前输入产生，不要求相加为 1。$\alpha_t = 1$ 时退化为 DeltaNet，接近 0 时可以快速清除旧状态。

每个 head 用一个标量衰减整个状态，所有方向都按相同比例缩小。如果希望少保留某些方向、多保留另一些方向，就需要更细的控制。

三、KDA：每个 Key 通道独立衰减

Gated DeltaNet 用同一个系数衰减整个状态：想让第一行少保留一些，第二行也会一起缩小。**Kimi Delta Attention（KDA）把这个标量改成一组系数，每一行可以采用不同的保留比例。**

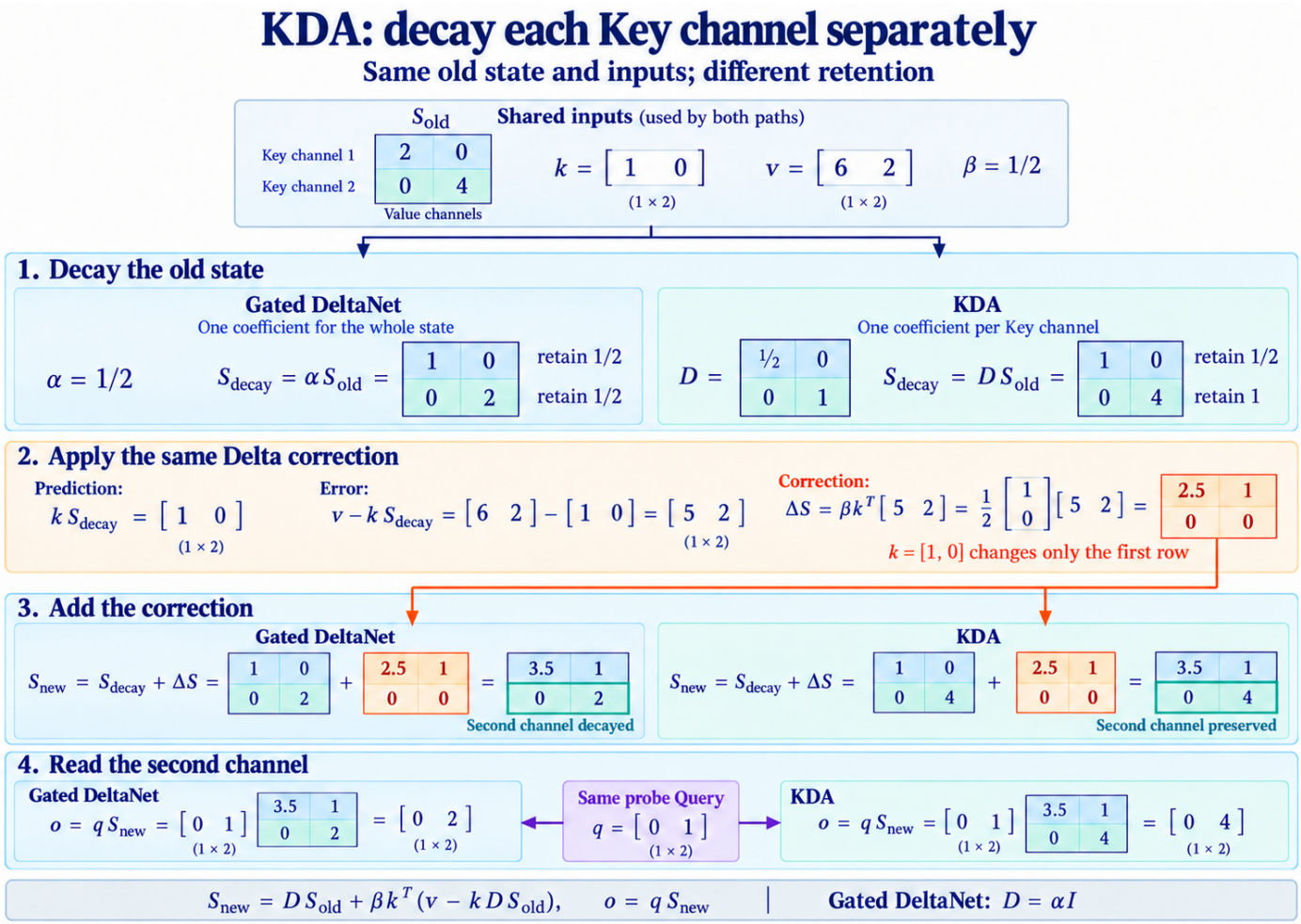
这里的"Key 通道"就是 Key 的一个分量。状态 S 的形状为 $d_k \times d_v$，每一行对应一个 Key 通道，每一列对应一个 Value 通道；计算 `kS` 时，用 Key 的各个分量加权组合这些行。因此，给第 i 个 Key 通道设置保留系数 $\alpha_{t,i}$，就是把状态的第 i 行整体乘以这个系数。

把这些系数放在对角线上，得到 D_t 。KDA 仍然先衰减、再按差额修正，只是把 Gated DeltaNet 中的标量 α_t 换成了对角矩阵 D_t ： 论文 §3

$$
\begin{aligned}
D_t &= \mathrm{diag}(\alpha_{t,1}, \dots, \alpha_{t,d_k}), \\
S_t &= D_t S_{t-1} + \beta_t k_t^\top \left(v_t - k_t D_t S_{t-1}\right), \\
o_t &= q_t S_t .
\end{aligned}
$$

$D_t S_{t-1}$ 分别缩放旧状态的各行；括号中仍是“当前 Value − 衰减后的预测”，外积修正和 Query 读取都沿用 DeltaNet 的方式。保留系数由当前输入产生，状态矩阵的大小不变。所有通道采用相同比例时，KDA 就退化为 Gated DeltaNet。

沿用 DeltaNet 图中的 S_1 ，两边用相同的 Key、Value 和 β 更新。Gated DeltaNet 将两行都乘 1/2；KDA 则让第一行乘 1/2，第二行乘 1：



本次 Key 为 `[1,0]`，修正只落在第一行，两边都得到 `[3.5,1]`。**区别在没有参与修正的第二行：Gated DeltaNet 将它从 `[0,4]` 衰减为 `[0,2]`，KDA 可以把它保留为 `[0,4]`。** 用同一个 Query `[0,1]` 读取，就能看到这部分信息保留程度的差异。

这些通道是学习出的特征维度，每一行仍混合了多条历史的贡献；逐通道衰减控制的是各行保留多少，不是逐个选择历史 token。

四、分块执行：块内矩阵计算，块间传递状态

Decode 每次只处理一个新 token，可以直接读取旧状态、更新并输出。训练和 Prefill 一次输入很多 token，如果仍逐个更新，大量小计算和连续的状态依赖就难以充分利用 GPU。**分块执行将一段 token 的计算组织成矩阵运算，在块之间传递状态。**

从逐 token 更新到块内方程

一个块包含 C 个 token，按块内位置 1 到 C 编号，入口状态记为 S_{in} 。沿用前文的行向量约定，`q_i`、`k_i` 的形状为 $1 \times d_k$ ，`v_i` 的形状为 $1 \times d_v$ ，状态的形状为 $d_k \times d_v$ 。

从 DeltaNet 的逐 token 更新开始，将每次写入的修正向量记作 r_i ：

$$
\begin{aligned}
r_i &= \beta_i(v_i - k_i S_{i-1}),\\
S_i &= S_{i-1} + k_i^\top r_i,\\
o_i &= q_i S_i.
\end{aligned}
$$

从块入口累积此前的修正，当前位置更新前的状态就是：

$$
S_{i-1} = S_{\text{in}} + \sum_{j=1}^{i-1} k_j^\top r_j.
$$

代入修正公式，消去各位置的中间状态：

$$
r_i = \beta_i \left(v_i - k_i S_{\text{in}} - \sum_{j=1}^{i-1} (k_i k_j^\top) r_j \right),
$$

$$
r_i + \beta_i \sum_{j=1}^{i-1} (k_i k_j^\top) r_j = \beta_i (v_i - k_i S_{\text{in}}).
$$

$k_i k_j^\top$ 是标量，表示第 j 次写入对当前 Key 读取的影响。**入口状态与此前修正共同决定当前修正；这个依赖只涉及块内更早的位置。**

将整个块写成下三角方程

将各 token 的行向量堆成矩阵，定义：

$$
K = \begin{bmatrix} k_1 \\ \vdots \\ k_C \end{bmatrix}, \qquad V = \begin{bmatrix} v_1 \\ \vdots \\ v_C \end{bmatrix}, \qquad R = \begin{bmatrix} r_1 \\ \vdots \\ r_C \end{bmatrix},
$$

$$
B = \operatorname{diag}(\beta_1, \dots, \beta_C), \qquad L = \operatorname{tril}(KK^\top, -1).
$$

K 的形状为 $C \times d_k$ ，V、R 的形状为 $C \times d_v$ ，B、L 的形状为 $C \times C$ 。L 只保留严格下三角部分。三个 token 时，它是：

$$
L = \begin{bmatrix} 0 & 0 & 0 \\ k_2 k_1^\top & 0 & 0 \\ k_3 k_1^\top & k_3 k_2^\top & 0 \end{bmatrix}.
$$

上一节的全部方程合起来，就是：

$$R + BLR = B(V - KS_{\text{in}}), \qquad (I + BL)R = B(V - KS_{\text{in}}).$$

矩阵没有消除依赖，而是把依赖写在下三角部分。 B 左乘 L，按各行的 β 缩放此前修正的贡献；当前位置和未来位置不在 L 中。

将入口状态单独留下

$I + BL$ 的对角线全为 1，因此可逆。定义 T，再把本块输入与入口状态分开：

$$\begin{aligned} T &= (I + BL)^{-1}B, \\ R &= TV - TKS_{\text{in}}, \\ U &= TV, \qquad W = TK, \\ R &= U - WS_{\text{in}}. \end{aligned}$$

T、U、W 都只由本块的 Key、Value 和 β 决定，可以在入口状态尚未确定时准备。 U 是变换后的 Value 矩阵，W 描述入口状态对修正量的影响；结合入口状态后，才得到实际写入的修正矩阵 R。

推出块末状态

将整个块的修正累积进入口状态：

$$\begin{aligned} S_{\text{out}} &= S_{\text{in}} + \sum_{i=1}^{C} k_i^\top r_i \\ &= S_{\text{in}} + K^\top R \\ &= S_{\text{in}} + K^\top (U - WS_{\text{in}}). \end{aligned}$$

这就是一整块 token 对状态的更新。下一块以这个 S_{out} 作为自己的入口状态。

推出块内所有位置的输出

当前位置读取的是包含本次修正的状态，因此：

$$\begin{aligned} S_i &= S_{\text{in}} + \sum_{j \le i} k_j^\top r_j, \\ o_i &= q_i S_{\text{in}} + \sum_{j \le i} (q_i k_j^\top) r_j. \end{aligned}$$

将 Query 堆成矩阵 Q，令 M 为包含对角线的下三角因果掩码：当前位置及此前位置为 1，未来位置为 0。将全部输出堆成矩阵 O，得到：

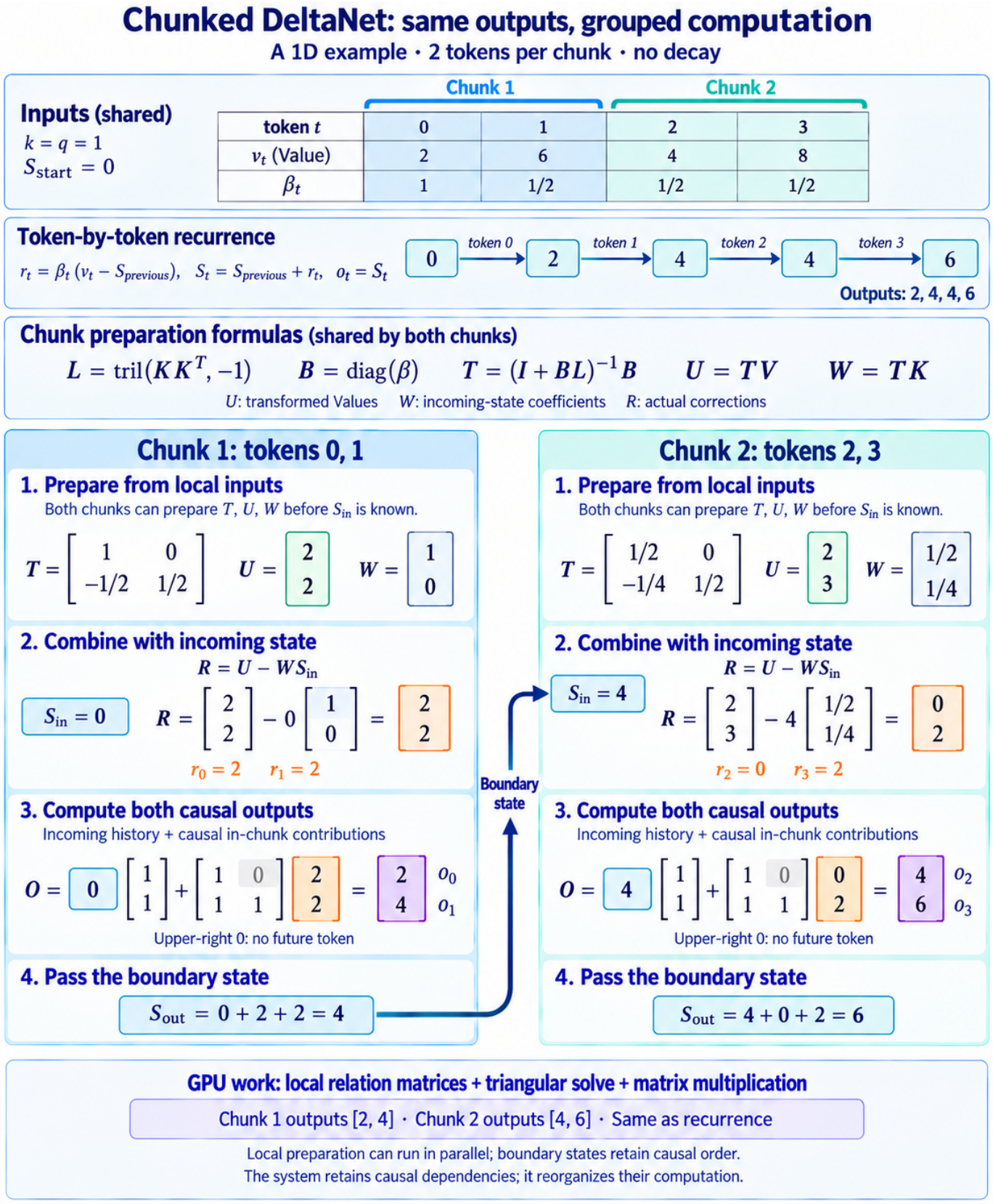
$$\begin{aligned} O &= QS_{\text{in}} + \Big((QK^\top) \odot M\Big) R \\ &= QS_{\text{in}} + \Big((QK^\top) \odot M\Big)(U - WS_{\text{in}}). \end{aligned}$$

$\odot$ 表示逐元素相乘。Q 的形状为 $C \times d_k$，O 的形状为 $C \times d_v$：每一行仍是一个位置自己的输出，并没有让所有 Query 都读取块末状态。

至此，逐 token 的更新被改写为**块内变换 T、U、W，块末状态更新，以及整块因果输出计算**。上面的形式对应并行 DeltaNet 论文 §3.2，公式 8–11，统一采用本课的行向量约定。

代入一维算例

图中用一维 DeltaNet 展示这个过程：Key、Query 都为 1，每两个 token 分成一块。上面逐 token 更新，下面分块计算，两种路径得到相同的输出：



图中 token 按全局位置 0 到 3 编号，橙色 R 是实际修正矩阵，各行记作 r。每块的计算分成四步：

1. **准备 T、U、W。** 由本块的 K、V 和 β 计算；两块都可以在入口状态尚未确定时完成这一步。
2. **结合入口状态得到 R。** 第一块的入口状态为 0，R 与 U 相同。第二块的 U 为 `[2,3]ᵀ`、W 为 `[1/2,1/4]ᵀ`，入口状态为 4，因此 `R = U − 4W = [0,2]ᵀ`。
3. **生成各位置的输出。** 将入口状态与截至当前位置的修正相加。输出计算中的因果矩阵右上角为 0，第一个位置不会用到第二个位置的修正。

4. **传递块末状态。** 第一块传出 4，第二块将它作为入口状态继续计算，最终传出 6。图中的边界箭头连接到第二块的状态结合步骤。

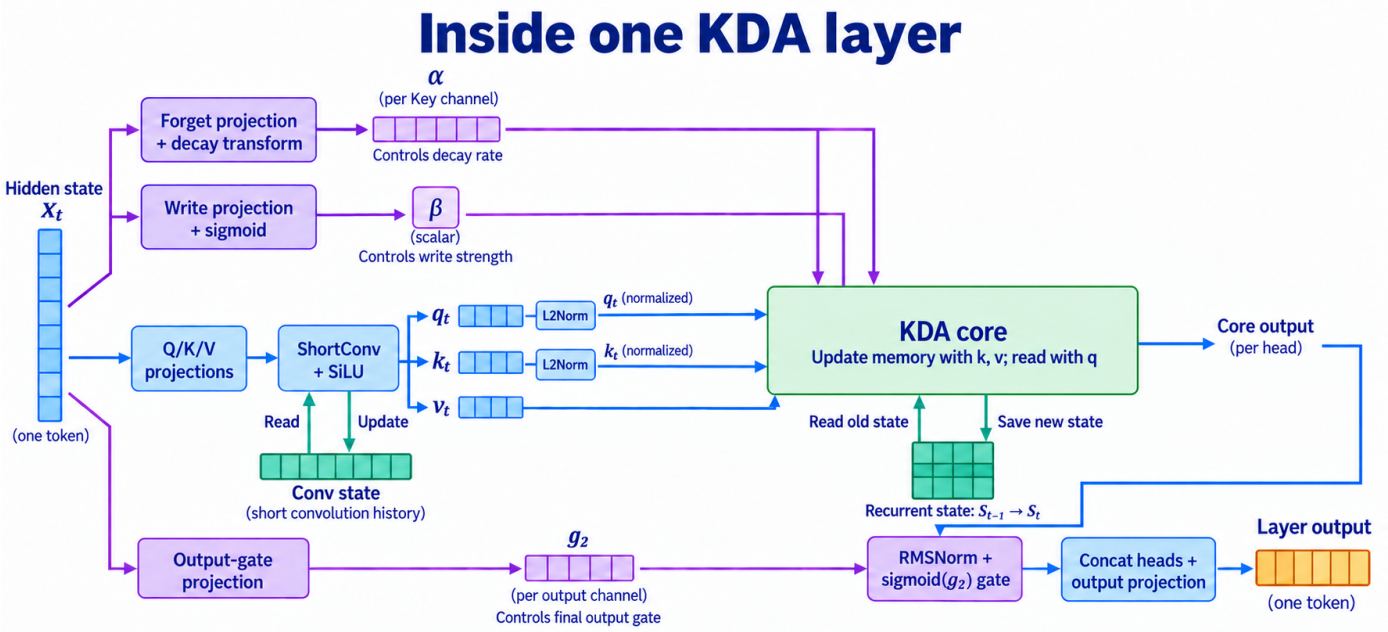
分块保留了 token 之间的依赖，但把依赖写进块内关系矩阵，通过三角求解和矩阵运算处理。 这样可以批量计算块内关系和输出，减少逐 token 执行小操作、保存完整中间状态的开销。并行 DeltaNet 论文 §3.2

KDA 还需在块内关系和状态传递中计入逐通道衰减。vLLM 的 KDA 分块实现 依次计算块内关系、求解三角依赖、传递块状态和生成输出；Decode 则继续使用逐 token 的递推路径。

五、vLLM 中的 KDA 执行与状态保存

以下对应 **vLLM v0.25.1**，KDA 层由 `KimiGatedDeltaNetAttention` 实现。

KDA 层的计算流程



Schematic tensor shapes. Projections, short convolution and output processing complete the KDA layer.

输入隐藏向量先投影成 Q/K/V，经过短因果卷积和 SiLU；Q/K 再做 L2 归一化，进入 KDA 计算。**K/V 修正状态，Query 读取新状态。** 读出结果经过 RMSNorm、输出门控和输出投影，回到模型的隐藏维度。源码

图中的三个门控控制不同位置：由 `g1` 转换得到的逐通道衰减作用于旧状态，`beta` 控制差额写入强度，`g2` 控制 RMSNorm 后的输出。

Prefill 与 Decode 的执行方式

`_forward()` 根据当前批次是否包含 Prefill，选择不同算子：

路径	算子	状态处理
包含 Prefill	`chunk_kda_with_fused_gate()`	读取各请求的入口状态，分块计算并保存块末状态
纯 Decode	`fused_recurrent_kda()`	按请求状态索引读取缓存，逐 token 递推并原位更新

在 Decode 的递推内核中，计算顺序就是前面的公式：**衰减旧状态 → 计算预测差额 → 写入修正 → Query 读取。**

保存两份状态

KDA 层的 `kv_cache` 保存的是 `conv_state` 和 `recurrent_state`：

- `conv_state`：保存短因果卷积所需的最近 Q/K/V 投影输入。
- `recurrent_state`：保存前文推导的状态矩阵 S，持续吸收历史信息。

每个请求通过状态索引找到自己的两份缓存，下一轮计算在其上继续更新。